📊 Relatório de Continuidade - Dashboard Freelancer

Projeto: PreJud SaaS | **Data:** 20/03/2026 | **Sessão:** Retomada de Continuidade | CONCLUÍDO

🎯 Objetivo da Sessão

Finalizar a integração do dashboard do freelancer com dados reais do backend, corrigindo inconsistências de datas e garantindo que os acordos sejam abertos pela rota pública `/p/{protocol}?t={clientAccessToken}` quando disponível.

📋 Resumo Executivo

A sessão teve como foco a **correção de bugs críticos** relacionados à conversão de timestamps do Firestore e a **integração completa** do dashboard com a Cloud Function `getFreelancerDashboard`. Foram identificadas e resolvidas inconsistências no formato de dados entre a Cloud Function e o frontend.

Item	Status	Detalhes
Integração com Cloud Function	✓ Funcionando	Hook useFreelancerDashboard conectado
Conversão de Timestamps	✓ Corrigido	Suporte a {_seconds, _nanoseconds}
Links Públicos de Acordos	✓ Implementado	Rota /p/{protocol}?t={token}
Formatação de Valores	✓ Corrigido	Formato BRL R$ 0,00
Build de Produção	✓ Sucesso	Sem erros TypeScript

🔧 Arquivos Modificados

TS **`page.tsx`**
frontend/app/(dashboard)/dashboard/page.tsx PRINCIPAL

Alterações Realizadas:

- ✓ Integração do hook `useFreelancerDashboard` para consumo de dados reais
- ✓ Implementação de estados de loading e erro com componente Loader2

- ✓ Função `getAcordoLink()` para construção de links públicos com fallback para rota interna
- ✓ Correção na renderização de datas com suporte a múltiplos formatos (string ISO, Date, Timestamp Firestore)
- ✓ Formatação de valores monetários em BRL
- ✓ Tratamento de campos opcionais (title/titulo, clientName/cliente)
- ✓ Mensagem de empty state quando não há acordos

```
// Função para construir link do acordo (público ou interno) const getAcordoLink = (agreement: Agreement): string => { if (agreement.protocol && agreement.clientAccessToken) { return `/p/${agreement.protocol}?t=${agreement.clientAccessToken}`; } // Fallback para rota interna se não tiver protocol/token return `/dashboard/acordos/${agreement.id}`; };
```

TS **dashboardService.ts**
frontend/services/dashboardService.ts
CRÍTICO

Alterações Realizadas:

- ✓ Correção da função `convertTimestamp()` para suportar formato `{_seconds, _nanoseconds}`
- ✓ Adição de fallback para formato `{seconds, nanoseconds}` (sem underscore)
- ✓ Preservação de `protocol` e `clientAccessToken` no mapeamento de agreements
- ✓ Conversão recursiva de timestamps na timeline
- ✓ Função `formatValue()` para formatação monetária
- ✓ Remoção de caracteres UTF-8 inválidos em comentários

Problema Crítico Identificado

A Cloud Function retorna timestamps no formato `{_seconds: 1775001600, _nanoseconds: 0}`, mas o código original esperava `{seconds: ...}` (sem underscore). Isso causava "Invalid Date" em todos os acordos exceto o primeiro.

Solução Implementada

Adicionada verificação dupla na função convertTimestamp para suportar ambos os formatos:

```
// Timestamp do Firestore como objeto {_seconds, _nanoseconds} if (typeof timestamp ===
'object' && timestamp !== null && '_seconds' in timestamp) { const seconds = (timestamp as {
_seconds: number }).._seconds; return new Date(seconds * 1000); }
```

🐛 Bugs Corrigidos

Bug	Causa Raiz	Solução
"Invalid Date" no Prazo	Formato {_seconds} não reconhecido	Atualização do convertTimestamp()
"Invalid Date" no Valor	Campo numérico sendo tratado como data	Formatação condicional por tipo
Erro UTF-8 no build	Caractere acentuado em comentário	Remoção de acentos em comentários
Hook sem freelancerId	Chamada sem parâmetro obrigatório	Passar user?.uid do AuthContext
Links para rota interna	Não usava protocol/token	Função getAcordoLink() com fallback

🏗️ Arquitetura da Integração

Fluxo de Dados

Cloud Function (Node.js) → dashboardService.ts → useFreelancerDashboard hook → Dashboard Page (React) → UI Components

Firebase Functions | TypeScript | React | Next.js 16 | Tailwind CSS | Firestore

📊 Dados Processados

O dashboard consome e exibe os seguintes campos dos acordos:

- ✓ **Identificação:** id, title, protocol, hash
- ✓ **Participantes:** freelancerName, clientName, clientEmail
- ✓ **Comercial:** value (formatado como BRL), deadline (convertido para Date)
- ✓ **Status:** status (draft, confirmed, etc.)
- ✓ **Acesso Público:** protocol, clientAccessToken

✓ **Timeline:** Array de eventos com datas convertidas

✅ Validação Final

Testes Realizados

- ✓ Build de produção executado sem erros TypeScript
- ✓ Emulador Firebase Functions iniciado e respondendo
- ✓ Dashboard carregando lista de acordos do freelancer autenticado
- ✓ Formatação correta de valores (R$ 100,00)
- ✓ Conversão correta de datas (29/04/2026)
- ✓ Links "Abrir acordo" direcionando para rota pública quando disponível
- ✓ Fallback para rota interna quando protocol/token ausentes

📝 Pendências e Próximos Passos

Item	Prioridade	Descrição
Cobranças	Média	Integrar dados reais de cobranças (atualmente mockado)
Histórico	Média	Buscar timeline de atividades do backend
Ações Rápidas	Baixa	Implementar funcionalidades dos botões (Gerar cobrança, Emitir notificação)
Deploy	Alta	Deploy da Cloud Function para produção

🔐 Considerações de Segurança

- ✓ Rota pública `/p/{protocol}` valida token de acesso antes de exibir dados sensíveis
- ✓ Dashboard só exibe acordos do freelancer autenticado (filtrado por freelancerId)
- ✓ Cloud Function verifica autenticação Firebase antes de retornar dados

---

Relatório gerado em: 20/03/2026 às 11:48

Desenvolvedor: Assistente de IA | **Revisão:** Continuidade de Projeto

Este documento é confidencial e destinado apenas para uso interno da equipe PreJud.